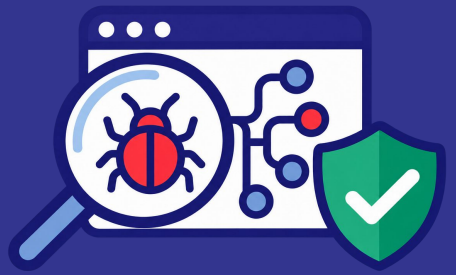
بسم الله الرحمن الرحیم

Data and Network Security

SESSION 08

Vulnerability Discovery and Program Analysis



Solmaz Salimi
Sharif University of Technology

s.salimi@sharif.edu

Last lecture and today's lecture

Session 7	CFI, PAC, MTE, and the bugs mitigations leave behind
Part 1	Coverage-guided fuzzing and test harnesses
Part 2	Static analysis and dynamic taint tracking
Part 3	Symbolic execution and path constraints
Part 4	Root-cause analysis, patching, and regression tests
Checkpoint	Choose a fix for a length-controlled copy







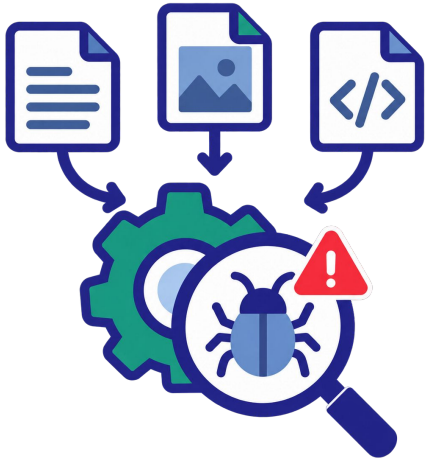

Evidence in vulnerability discovery

A failure needs an explanation before it becomes a security finding.

Reachability	**Can attacker-controlled input reach the faulty operation?**
Failure	**Which invariant does the operation violate?**
Impact	**What security property can the violation affect?**
Repair	**Which change restores the invariant for related inputs?**

A crash is evidence of a failure. Exploitability depends on the surrounding system and threat model.

Part 1: Coverage-guided fuzzing and reliable test harnesses



Running example: a length-controlled record

```
if (size < 3) return;
if (data[0] != 'M' ||
    data[1] != 'G') return;
size_t n = data[2];
if (n > size - 3) return;
char out[16];
memcpy(out, data + 3, n);
consume(out, n);
```

Header — two magic bytes and one length byte

Source — the payload must contain n bytes

Destination — out can hold only 16 bytes

The source-length check does not establish the destination's capacity.

Input-generation strategies

Mutation

Alter existing sample inputs with byte flips, insertions, and boundary values

```
MG | 05 | HELLO    Original
MG | 05 | HXLLO    Byte flip
MG | 00 | HELLO    Change length
MG | FF | HELLO    Boundary value
MG | 06 | HELLO!   Insert byte
```

needs a useful corpus

Generation

Create inputs from a format or grammar model

```
Record:
  Magic   = "MG"
  Length  = 1 byte
  Payload = Length bytes

MG | 00 |
MG | 03 | ABC
MG | 10 | AAAAAAAAAAAAAAAA
MG | 11 | AAAAAAAAAAAAAAAAA
```

models structure

Coverage guidance

Use execution feedback to prioritize inputs that reach new behavior

Execute candidate
↓
Measure new coverage
↓
Retain useful inputs

guides either strategy

Generation and mutation describe how inputs are built. Coverage describes how the search receives feedback.

The coverage-feedback loop



Repeat with the updated corpus. Coverage guides exploration but does not prove that a path is safe.

A harness exposes the code under test

```
extern "C" int
LLVMFuzzerTestOneInput(
  const uint8_t *data,
  size_t size) {
  parse_record(data, size);
  return 0;
}
```

Scope	call the real parser
Inputs	tolerate empty and malformed data
State	reset state between iterations

A fast, repeatable harness improves the search and makes failures easier to reproduce.

Fuzzing Oracles



The oracle decides which observed behavior counts as a failure.

Crash	**A fatal signal or unexpected termination**
Sanitizer	**A report such as an out-of-bounds access or use-after-free**
Invariant	**An assertion or security-specific consistency check**
Differential	**A disagreement between implementations with equivalent semantics**

ASan makes many memory errors observable before they become an unexplained crash.

Worked fuzzing trace for the record parser

Input	Path reached	Observation
Wrong magic	Header rejects input	Parser returns
n = 16, size = 19	Valid header and copy	16-byte destination fits
n = 17, size = 20	Same copy location	Invalid destination write

The n = 17 input can reuse the same copy path as n = 16. The oracle detects the invalid write.

Fuzzing blind spots

The harness and search budget bound what fuzzing can observe.

Barriers

Magic values, checksums, and complex input formats
Example: random input fails the MG header check.

State

Authentication, protocol sequences, and persistent state
Example: WRITE is rejected until LOGIN succeeds.

Oracle

Silent logic errors can escape a crash-only check
Example: Bob is credited but Alice is not debited.

Budget

Incomplete environments and limited execution time
Example: an interrupt-only path is never exercised.

No failure found means no failure observed under this harness and budget.

Part 2: Static analysis and dynamic taint tracking



Static analysis reasons about modeled executions



Code model

Build a representation of control flow and data flow

no target run required

State reasoning

Track possible values, aliases, and constraints

analysis assumptions

Evidence

Report a suspicious operation and the path or condition that could reach it

review the warning

A useful result states the checked property and the assumptions of the analysis.

False alarms and missed bugs

FALSE POSITIVE

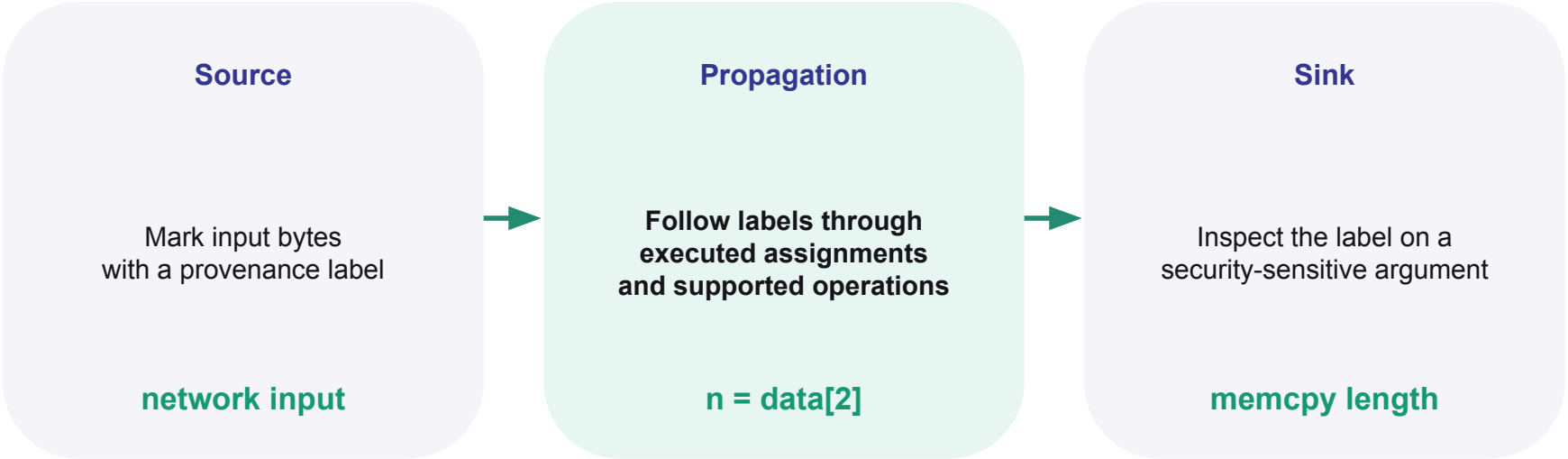
The analysis reports a bug that cannot occur in the relevant execution model.

FALSE NEGATIVE

A real bug occurs in behavior that the analysis fails to report.

Approximations and incomplete models influence which warnings need validation and which bugs remain hidden.

Dynamic taint tracks selected data dependencies



The label shows where data came from. A separate policy decides whether its use is unsafe.

Validation must match the sink’s contract



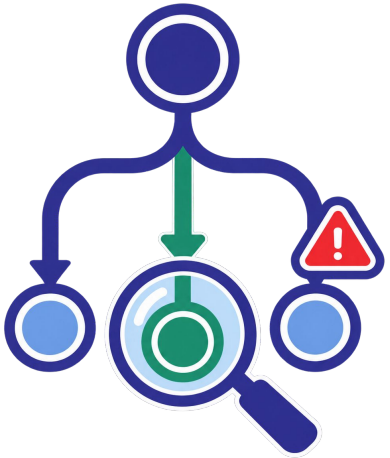
```
n = data[2];
if (n > size - 3) return;

/* n still comes from input */
memcpy(out, data + 3, n);
/* need n <= sizeof(out) */
```

Source	n has input provenance
Existing check	enough readable payload bytes
Missing check	enough writable destination bytes

Passing one validation check does not establish every property needed at the sink.

Part 3: Symbolic execution and path constraints



What replaces a concrete input?

Concrete bytes → symbolic variables → a solver-produced test input

CONCRETE INPUT

```
data[0] = 'M'
data[1] = 'G'
data[2] = 0x05
size = 8; n = 5
```

One fixed test

→

SYMBOLIC INPUT

```
data[0] = B0
data[1] = B1
data[2] = N
size = S; n = N
```

Unknown bytes and length

→

SOLVER'S CANDIDATE

```
B0 = 'M'; B1 = 'G'
N > 16; N <= S-3
N = 17; S = 20
```

`MG | 11 | A × 17`

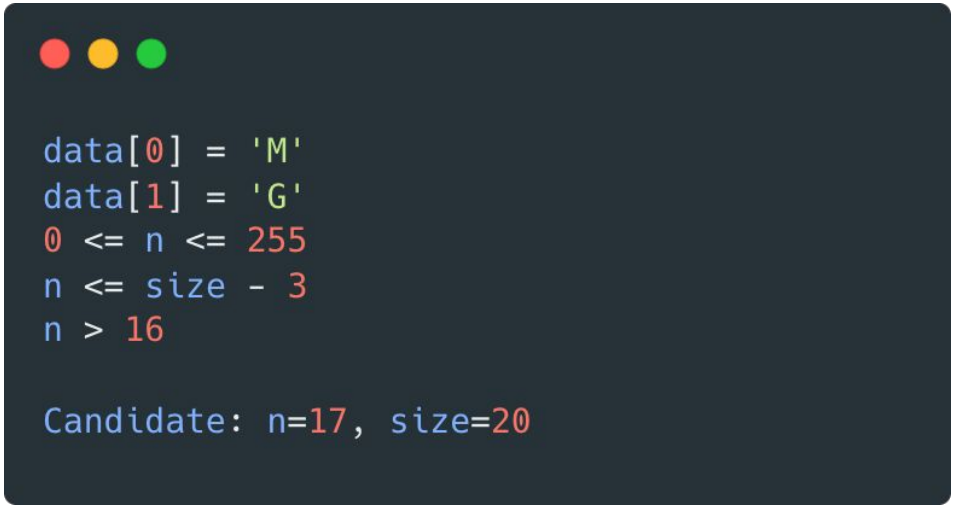
STATIC SYMBOLIC EXECUTION

Start with symbolic values (B0, B1, N, S).
Explore feasible paths in a program model.
Use a solver to create concrete tests.

DYNAMIC / CONCOLIC EXECUTION

Run a concrete seed (e.g., n = 5).
Track symbolic constraints along that run.
Solve for another concrete test input.

A solver can propose a concrete failing input



Symbols — input bytes and input length

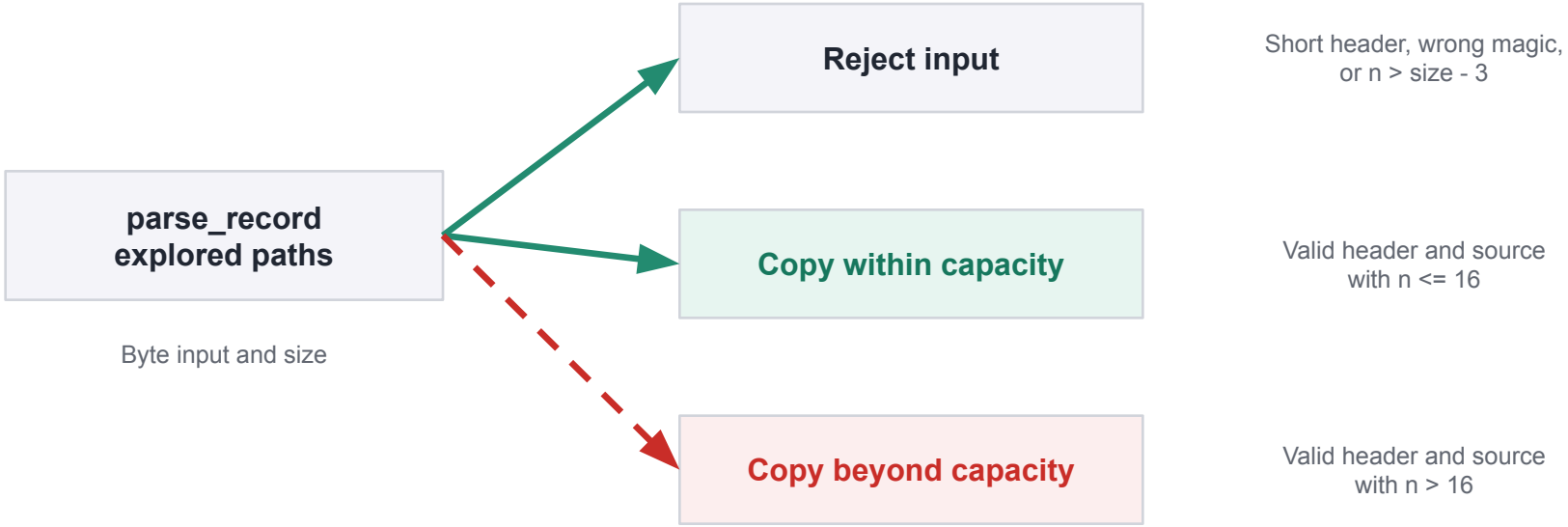
Path — constraints select an unsafe copy

Replay — test the candidate on the real build

The solver satisfies the modeled conditions. A concrete replay checks whether that model matches the target.

Three path classes in the record parser

The bug path needs both a valid source extent and an excessive destination length.



A path condition records decisions. The solver searches for an input satisfying those decisions.

Symbolic execution has practical limits

The engine explores a model under finite resource limits.

Paths	Branches and loops multiply execution states
Models	Files, system calls, and libraries need usable models
Solver	Complex constraints may exhaust time or memory
Bounds	Input-size and loop limits restrict explored behavior

Generated tests provide evidence. Unexplored behavior remains outside the result.

The methods answer different questions

Method	Main evidence	Important limitation
Fuzzing	Concrete inputs and observed failures	Harness, oracle, and test budget
Static analysis	Potential violation in a code model	Approximations and incomplete models
Dynamic taint	Input provenance on an executed path	Label policy and propagation coverage
Symbolic execution	Path conditions and candidate inputs	Path explosion and environment models

The categories overlap. A practical workflow combines their evidence.

Part 4: Root-cause analysis and security patching



A reproducer begins the root-cause analysis



1 Reproduce

Record input, build, and environment for a repeatable failure

retain evidence

2 Minimize

Remove irrelevant bytes while preserving the same failure

smaller explanation

3 Explain

Trace the invalid operation back to the violated validation or lifetime rule

root cause

Minimization simplifies the witness. The patch must address the vulnerable operation beyond that one witness.

A patch restores both copy bounds

```
if (size < 3) return;
if (data[0] != 'M' ||
    data[1] != 'G') return;
size_t n = data[2];
char out[16];
if (n > sizeof(out)) return;
if (n > size - 3) return;
memcpy(out, data + 3, n);
consume(out, n);
```

Destination	n <= sizeof(out)
Source	n <= size - 3
Order	check header before subtraction

The patch rejects all excessive lengths while preserving valid records up to 16 bytes.

Regression tests cover the boundary and valid use

Case	Input condition	Expected patched behavior
Empty payload	Valid header, n = 0	Accept byte-counted record
Exact capacity	n = 16, 16 payload bytes	Accept without invalid write
Over capacity	n = 17, 17 payload bytes	Reject
Truncated source	n = 16, only 15 payload bytes	Reject
Wrong magic	Header does not start with MG	Reject

The original failure, nearby lengths, malformed records, and valid behavior all need tests.

Checkpoint: a patch that blocks one sample

Assume valid magic and enough payload

```
char out[16];
if (n == 17) return;
memcpy(out, data + 3, n);
consume(out, n);
```

1 Does this repair the destination-bound invariant?

2 Which nearby input would still violate it?

3 Which valid case must the real patch preserve?

Block all n > 16, retain the source-length check, and keep n = 16 valid.

References

COURSES	Stanford CS 155, Lecture 4, Brandon Azad, 2026. Berkeley CS 161 Software Security, 2020: fuzz-testing introduction.
UW LAB 4	CSE 484 / M584, Lab 4: Root-Cause Analysis and Patching, Spring 2025. Evidence, general fixes, and functionality tests.
TOOLS	LLVM libFuzzer, AddressSanitizer, Clang Static Analyzer, and DataFlowSanitizer documentation. KLEE Tutorial One.
PRIOR TERM	Control-hijacking slides: fuzzing for overflows and memcpy bounds checks. No standalone analysis deck in the supplied archive.

Next: operating-system isolation and sandboxing



Session 9 examines how a compromised process's authority is constrained.

setuid	**Identity, authority, and requested operations**
TOCTOU	**State changes between a check and its use**
Sandboxing	**System-call and resource boundaries**
Isolation	**Processes, containers, virtual machines, and SFI**

Today's review: find a witness, explain the invariant, repair the bug, and test related behavior.